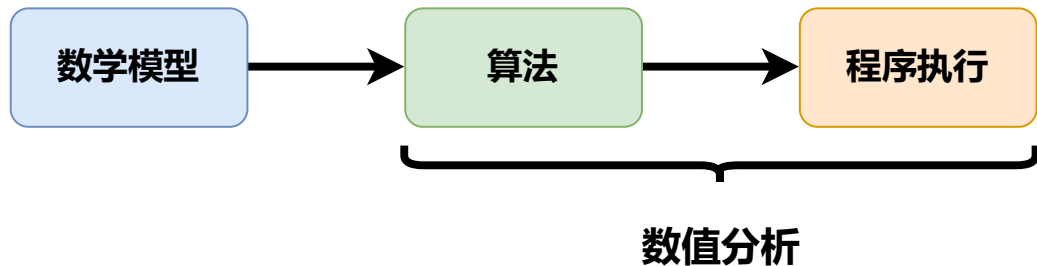


数值分析与科学计算引论

主 讲 人：金永文

2025.2.21

数值分析的对象、作用与特点



好的算法：可靠的理论分析 + 好的计算复杂性 + 数值实验

误差来源与分类

- 模型误差：理论建模不精确带来的误差。

例：经典力学与相对论

牛顿第二定律： $F = m \frac{dv}{dt}$

相对论： $m = \frac{m_0}{\sqrt{1 - \frac{v^2}{c^2}}}$, $F = \frac{dp}{dt} = \frac{dm}{dt} \cdot v + m \cdot \frac{dv}{dt}$

在低速情况下这样的误差可以忽略。

- 观测误差：由测量的不精确性带来的误差。
- 截断误差（方法误差）：近似解与精确解之间的误差。

例：采用泰勒多项式近似函数

$$P_n(x) = f(0) + \frac{f'(0)}{1!}x + \frac{f''(0)}{2!}x^2 + \cdots + \frac{f^{(n)}(0)}{n!}x^n$$

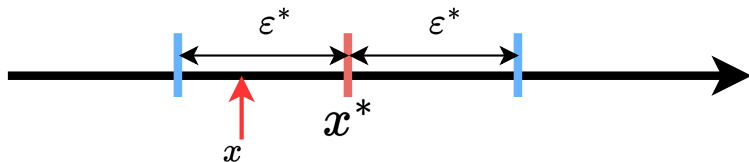
相应的截断误差为： $R_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}x^{n+1}$, $\xi \in (0, x)$ （拉格朗日型余项）

误差与有效数字

定义 1

设 x 为准确值, x^* 为 x 的一个近似值, 称 $e^* = x^* - x$ 为近似值的绝对误差, 简称误差.

$|e^*|$ 的上限为 ε^* , 称为绝对误差限, 工程上记为: $x = x^* \pm \varepsilon^*$.



注: e^* 理论上讲是唯一确定的, 可能取正, 也可能取负。 $\varepsilon^* > 0$ 不唯一, 当然 ε^* 越小越具有参考价值。

误差与有效数字

只需要误差就够了吗？

例

1. 小明身高 2 米左右（实际 1.65 米，误差 0.35 米），他今天走了大约 10km（实际 10.1 千米，误差 100 米）。
2. 小张今年 18 岁左右（实际 30 岁，误差 12 岁），他有个北宋时期的碗，大约距今有 1000 年（实际 1025 年，误差 25 年）。

相对误差：近似值的误差 e^* 与准确值 x 的比值

$$e_r^* = \frac{e^*}{x} = \frac{x^* - x}{x}.$$

实际常采用 $e_r^* = \frac{x^* - x}{x^*}$.

相对误差限： $\varepsilon_r^* = \frac{\varepsilon^*}{|x^*|}$

误差与有效数字

定义 2

若近似值 x^* 的误差限是某一位的半个单位，该位到 x^* 的第一位非零数字共有 n 位，就说 x^* 有 n 位有效数字. 它可表示为

$$x^* = \pm 10^m \times (a_1 + a_2 \times 10^{-1} + \cdots + a_n \times 10^{-(n-1)}),$$

其中 $a_i (i = 1, 2, \dots, n)$ 是 0 到 9 中的一个数字， $a_1 \neq 0$ ， m 为整数，且

$$|x - x^*| \leq \frac{1}{2} \times 10^{m-n+1}.$$

$$\pi^* = 3.14159$$

$$\text{误差限: } e^* = 0.000005 \geq |\pi - \pi^*|$$



单位: 0.00001

误差与有效数字

例题 1

判断以下数字有几位有效数字：

0.0034

0.1000

0.35×10^5

0.350×10^5

例题 2

$\pi = 3.1415926535897932 \dots$, $\pi^* = 3.1415$ 。问： π^* 有几位有效数字？

误差与有效数字

定理 1

设近似数 x^* 表示为

$$x^* = \pm 10^m \times (a_1 + a_2 \times 10^{-1} + \cdots + a_l \times 10^{-(l-1)}),$$

其中 $a_i (i = 1, 2, \dots, l)$ 是 0 到 9 中的一个数字, $a_1 \neq 0$, m 为整数. 若 x^* 具有 n 位有效数字, 则其相对误差限

$$\varepsilon_r^* \leq \frac{1}{2a_1} \times 10^{-(n-1)};$$

反之, 若 x^* 的相对误差限 $\varepsilon_r^* \leq \frac{1}{2(a_1+1)} \times 10^{-(n-1)}$, 则 x^* 至少具有 n 位有效数字.

数值运算的误差估计

对于 $y^* = x_1^* + x_2^*$,

$$\varepsilon(y^*) \approx \varepsilon(x_1^*) + \varepsilon(x_2^*).$$

对于 $y^* = x_1^* \cdot x_2^*$,

$$\varepsilon(y^*) \approx |x_2^*| \cdot \varepsilon_1^* + |x_1^*| \cdot \varepsilon_2^*.$$

对于 $y^* = \frac{x_1^*}{x_2^*}$,

$$\varepsilon(y^*) \approx \frac{1}{|x_2^*|} \cdot \varepsilon_1^* + \frac{|x_1^*|}{|x_2^*|^2} \cdot \varepsilon_2^*.$$

设 $f(x)$ 是一元可微函数, x 的近似值为 x^* , 以 $f(x^*)$ 近似 $f(x)$, 其误差界记作 $\varepsilon(f(x^*))$ 。
假定 $f''(x^*)$ 与 $f'(x^*)$ 的比值不大, 则有

$$\varepsilon(f(x^*)) \approx |f'(x^*)|\varepsilon(x^*).$$

算法的数值稳定性

例

计算 $I_n = e^{-1} \int_0^1 x^n e^x dx (n = 0, 1, \dots)$ 并估计误差.

采用分部积分:

$$\int_0^1 x^n e^x dx = \int_0^1 (x^n e^x)' dx - \int_0^1 n x^{n-1} e^x dx = e - e \cdot n I_{n-1},$$

有

$$\begin{cases} I_n = 1 - n I_{n-1}, & n = 1, 2, \dots, \\ I_0 = e^{-1} \int_0^1 e^x dx = 1 - e^{-1}. \end{cases}$$

注意: 这一计算方式的误差也在逐步迭代:

$$E_n = -n E_{n-1} = (-1)^n n! E_0.$$

避免小分母

分母小会造成浮点溢出。

```
>> x=eps(0)
x =
    4.9407e-324
>> y=10
y =
    10
>> y/x
ans =
    Inf
```

避免相近二数相减

例 1

$a_1 = 0.12345$, $a_2 = 0.12346$, 各有 5 位有效数字。而 $a_2 - a_1 = 0.00001$, 只剩下一位有效数字。

几种经验性避免方法:

$$\sqrt{x + \varepsilon} - \sqrt{x} = \frac{\varepsilon}{\sqrt{x + \varepsilon} + \sqrt{x}}$$

$$\ln(x + \varepsilon) - \ln(x) = \ln\left(1 + \frac{\varepsilon}{x}\right)$$

$$\text{当 } |x| \ll 1: \quad 1 - \cos(x) = 2 \sin^2 \frac{x}{2};$$

$$e^x - 1 = x \left(1 + \frac{1}{2}x + \frac{1}{6}x^2 + \cdots \right)$$

避免大数吃小数

例 2

用单精度计算 $x^2 - (10^9 + 1)x + 10^9 = 0$ 的根。精确解为 $x_1 = 10^9$, $x_2 = 1$.

利用求根公式:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

在计算机内, 10^9 存为 0.1×10^{10} , 1 存为 0.1×10^1 . 做加法时, 两加数的指数先向大指数对齐, 再将浮点部分相加。即 1 的指数部分必须变为 10^{10} , 则: $1 = 0.0000000001 \times 10^{10}$. 两者相加, 保留 7 位有效数字:

$$10^9 + 1 = 0.1000000 \times 10^{10} + 0.0000000 \times 10^{10} = 0.1000000 \times 10^{10}.$$

采用求根公式, 有

$$x_1 = 10^9, \quad x_2 = 0.$$

尽量减少运算次数

例：秦九韶算法

给定 n 次多项式

$$p(x) = a_0x^n + a_1x^{n-1} + \cdots + a_{n-1}x + a_n, \quad a_0 \neq 0.$$

求 x^* 处的值 $p(x^*)$ 。

如果直接计算则需要 $\frac{n(n+1)}{2}$ 次乘法 ($1 + 2 + \cdots + n$), n 次加法。若采用

$$p(x) = (\cdots (a_0x + a_1)x + \cdots + a_{n-1})x + a_n,$$

可表示为

$$\begin{cases} b_0 = a_0, \\ b_i = b_{i-1}x^* + a_i, \quad i = 1, 2, \dots, n, \end{cases}$$

此算法称为秦九韶算法。